

Operating Systems

Tutorial Worksheet 06

Description. This tutorial worksheet covers the topic of monitors, process scheduling, and operating systems security.

1. Concept Understanding

- Semaphores are mutual-exclusive accessed shared variables used to set up synchronization among cooperative processes. Explain why semaphores cannot be used when cooperative processes are executing on different computers?
- Monitors is a high-level programming concept used to set up process synchronization. Monitors use semaphores in their low-level implementation but provide programmers with programming facilities to make synchronization coding less error-prone. Explain how monitors work?
- Pure monitors only provide mutual exclusion. Usually, for process synchronization, we also need to set up process ordering i.e., execute process P1 before P2. Which data structure do we need to do that? Explain how that data structure works?
- In monitors, we can use the `notify()` primitive to wake up a sleeping process from the waiting queue. This primitive appears to have two semantics. Why is that possible? Explain the difference between the two semantics.
- Explain under which circumstances, it is not correct to use the `notify()` primitive to wake up a sleeping process from the waiting queue, but the `notifyAll()` primitive should be used instead.
- What is the purpose of a process scheduling policy? What is the throughput, turnaround time, response time, and waiting time?
- Using round-robin scheduling, what is the advantage of choosing a short time quantum? What is the advantage of choosing a long time quantum?
- What is the different between an attack and an intrusion?
- What is the difference between an attack and a cyberattack?
- What are the fundamental security services? Think about other security services.
- Explain what is a rootkit, backdoor, and a trapdoor are.

2. Monitors for Process Synchronization

Consider the following producer-consumer problem: A customer enters a fastfood to grab $K \geq 1$ sandwiches and leave. If no sandwich is available ($N=0$), the customer waits in a queue. Customers can enter the shop at anytime and stand in the queue if no sandwich is available. A cooker, prepares $S \geq 1$ sandwiches at a time then randomly chose a customer to grab some sandwiches. The selected customer may not be the right customer, i.e., if the number of available sandwiches is less than what the customer needs, the customer needs to go back into the queue. Complete the following code so that customers can get their sandwiches and leave the store.

Monitor M

```
{
    Condition Variable:  Q;
    Integer:  N=0;
    Cook_sandwiches(Integer S)                Grab_sandwiches(Integer K)
    {                                           {
        ...                                     }
    }                                           }
}
```



```
Customer()                                Cook()
{                                           {
    while(true)
    {
        M.Grab_sandwiches(Integer K);        M.Cook_sandwiches(Integer S);
        Leave();                             Look_at_customers();
    }                                         }
}
```

You don't have to worry about the two functions: `Leave()` and `Look_at_customers()`.

3. Process Scheduling

Complete the following tables based on the adopted scheduling policy:

Caution. $0.5 + 0.5 = 1.40$ not 1.0.

- Apply the FCFS (First comes, First Served) scheduling algorithm to complete the following table then compute the average turnaround time.

Process PID	Arrival Time	Estimated Burst Time	Termination Time	Turnaround Time	Waiting Time
12581	10.00	2.00			
21458	10.10	1.00			
36944	10.25	0.25			

- Propose a different order of arrival for the three processes to improve the average turnaround time. What is your new average turnaround time?
- Considering the same example, apply the RR (Round Robin) scheduling algorithm with a quantum=0.5 to complete the following table then compute the average turnaround time.

Process PID	Arrival Time	Estimated Burst Time	Termination Time	Turnaround Time	Waiting Time
12581	10.00	2.00			
21458	10.10	1.00			
36944	10.25	0.25			

4. Operating Systems Security

- Classify the following attacks according to which security service they compromise (Just write the attack number in the table): (1) Tailgating someone to enter a restricted building (2) Shoulder surfing a student typing his/her password (3) Cutting Ethernet wires in the 1st floor corridor (4) Denying having sent an email to school office (5) Intercepting network traffic at school campus (6) Phishing new students (7) Vishing instructors (8) Unauthorized file destruction in classroom (9) Unauthorized file modification in classroom (10) Cracking root password for ensia's web Server (11) Sending blank emails to professors (12) Sending infected files to staff (13) Install a Spyware on somebody else computer (14) Sending a flood of ICMP packets to secretary computer (15) Sending a hoax to **social@ensia.edu.dz** telling them that there is free food in Amphi 06 (16) Stealing somebody else laptop (17) Plug a keylogger in a shared computer.

Attack on Authentication	Attack on Confidentiality	Attack on Availability	Attack on Integrity	Attack on Non-repudiation

- Explain the meaning of each security service mentioned in the above table.
- Consider the following C program:

```
#include <stdio.h>
#include <string.h>
#define BUFFER_SIZE 10
void main(int argc, char *argv[])
{
    char buffer[BUFFER_SIZE];
    if (argc < 2) return;
    else
```

```

    {
        strcpy(buffer, argv[1]);
        printf("run run ... !\n");
    }
}

```

This program appears to contain a vulnerability. Identify it and explain why it is a vulnerability. This vulnerability can be exploited by an attacker to generate a specific type of attack known as Buffer overflow on the stack. Explain how this works and how an attacker can take advantage of it to delete files in the target system.

- Consider the following C code snippet used for implementing a login procedure. Analyze the code snippet and identify the vulnerability and how it can be exploited to successfully login to the system in an unauthorized manner.

```

#include <stdio.h>
#include <string.h>

void main()
{
    char password[6] = {'e', 'n', 's', 'i', 'a', '\0'};
    char pass[6];
    char username[8];
    unsigned int miss = 0;

    printf("Username: ");
    gets(username);
    LABEL: printf("Password: ");
    gets(pass);

    if(strncmp(password, pass, 6) == 0)
    {
        printf("Logged in as %s ", username);
        execlp("/bin/sh", "/bin/./sh", null);
    }
    else
    {
        printf("Login failed");
        miss++;
        if(miss==3);
        else goto LABEL;
    }
}

```